Pranav Kunnamkudath Roy

Kattankulathur, Tamil Nadu, India | pranav.k.roy161@gmail.com | github.com/ReitardXd | reitardxd.me

Education

SRM Institute of Science and Technology — Kattankulathur, Tamil Nadu
B.Tech, Computer Science and Engineering — Computer Networking — *2023 — 2027*
- Cumulative GPA: 8.37/10

Mount St. Mary's School — New Delhi
Class XII, CBSE — *2022 — 2023*
- Percentage: 75.4%

Mount St. Mary's School — New Delhi
Class X, CBSE — *2020 — 2021*
- Percentage: 91.4%

Projects

Sked - Job Scheduler, Go, PostgreSQL, Redis, Docker, Kubernetes (github.com/ReitardXd/sked) 2026 — 2026
- Built a distributed job scheduler in Go with a Postgres-backed job store using `SELECT ... FOR UPDATE SKIP LOCKED` for concurrent-safe dequeue, verified via concurrent-claim testing with zero row overlap; added a Redis Streams queue decoupling scheduler dispatch from worker execution via consumer groups.
- Implemented Redis-based distributed leader election (`SET NX PX` with Lua-scripted lease renewal/release) so only one of multiple scheduler replicas dispatches at a time; verified failover by killing the active leader mid-run and confirming a standby took over within 35s with zero job loss or double-dispatch.
- Containerized all services (multi-stage Docker builds) and deployed the full stack to a local Kubernetes cluster (kind) with HPA-based worker autoscaling; instrumented API, scheduler, and worker with Prometheus metrics and structured JSON logging across the job lifecycle.

RWM — Rust Window Manager, Rust, x11rb, X11 (github.com/ReitardXd/rwm) 2025 — 2025
- Built a tiling window manager from scratch in Rust using x11rb raw X11 bindings, implementing the WM protocol end-to-end: SUBSTRUCTURE_REDIRECT takeover, ICCCM graceful close (WM_DELETE_WINDOW with XKillClient fallback), and EWMH title tracking.
- Designed a single-threaded, non-blocking event loop (200ms idle poll, 1Hz status-bar clock) with master-stack tiling across 9 workspaces; fixed a workspace-switching race via a pending-unmap queue — 1,057 LOC, 1 crate dependency, 0 unsafe blocks.

Seca - Social Reading Platform, React, Node.js, MongoDB, Groq API (github.com/ReitardXd/seca) 2025 — 2025
- Built a full-stack social reading platform (React 19/Vite/Tailwind client, Express 5/Mongoose API, MongoDB Atlas) with Google OAuth 2.0 and JWT authentication across 5 database models; deployed client on Vercel and server on Render.
- Integrated the Open Library API (with MongoDB caching) and Groq (Llama 3) for AI-generated quizzes with resilient JSON-parse recovery; implemented invite-code groups, reading streaks, and a per-group leaderboard — 2,136 LOC across 58 files.
- Live Demo: seca-xi.vercel.app

Technical Skills

- Languages: C++, C, Rust, Python, JavaScript, SQL, Bash/Shell
- Backend & Tools: FastAPI, Node.js, PostgreSQL, Docker, Git, GDB, Streamlit
- Systems & Networking: Linux (Arch), POSIX, TCP/IP, X11, Cisco DNAC API
- Graphics/Low-Level: OpenGL, GLFW
- Security: Vulnerability Research, Reverse Engineering, Binary Exploitation (HackTheBox, CTF)
- Competitive Programming: Codeforces Expert (1624 rating

Publications

- **Smart Sleep Posture Monitoring System Using Embedded Sensors and IoT** — International Research Journal on Advanced Engineering and Management, May 2025. Paper